# Flask Fundamentals

## Flask installation:

### locally:

1. Create the environment
   → python 3 -m venv.venv
2. activate it
   → source .venv / bin / activate
3. Install Flask
   → pip3 install flask
   → pip freeze > requirements.txt
4. When finished
   → deactivate
   → python 3 server.py

---

~~locally~~:

### globally:

python3 -m pip install --user flask

---

## Server.py

Here we'll set up all routes to handle requests

### Code

```
from flask import Flask
app = Flask (-- name --)
@ app. route ( '/ ')
def hello ( ):
    return " hello world "
if --name-- = "--main--":
    app. run (debug=True)
```

### What does the previous code do:

1. Import Flask
2. Make the app

   app = Flask(--name--)

NOTE --name--→ special built in variable

→ it's value depends on how the app runs.

So when I run `python app.py` then inside app.py → --name-- will be --main--

So --name-- is basically "--main--" if the file is executed directly



when you call it with python app.py

3. Create a route

   @app. route ('/') tells Flask:

   "When someone opens the homepage ("/")
   run the function below"

4. Run the app

   The if --name-- = "--main--" means:

   → only run the web server if the file is run directly not imported.

   → app. run (debug=True) starts the server.

   → debug = True means that Flask will run automatically when you change code.

NOTE The @ decorator associates this route with the function immediately following.

## Routes

Route:

- is like a variable name we assign to a request.
- The job of the route is to communicate to the server what kind of information the client needs.

---

- A route is like an address for your web page.
- It tells the server what to do when a user goes to a specific url.

Why do we need routes:

When someone visits a url:

The route decides:

1. What code to run.
2. What response to send back (text, HTML)

**A route = URL + action**

---

Every route has two parts:

1. HTTP method (GET, POST, PUT, PATCH)
2. URL

Code

```
@app.route('/hello/<name>')

def hello(name):
    print(name)
    return "hello" + name
```

example1

```
@route("hello/<username>/<id>")
def show_user(username, id)
    return username, id
```

---

NOTE

→ The localhost:5000 part of the route determine which server to call upon.

→ the part after /, tells the server which function should be invoked.

---

## Rendering Views

To render the content on the HTML page:

Create templates folder, and inside of it put the index.html

→ /hello_flask/templates/index.html

---

server.py

```
from -- import Flask, render_template
def hello_route():
    return render_template('index.html')
```

# Template Engines

## Template engines:

→ When we want to display dynamic content on a web page, we must use template engines.

To pass data along with the HTML.

→ The render_template function sends the HTML file and any accompanying data through the template engine.

→ which processes any code and produces the final HTML response to be sent to the client.

## Jija

{{ phrase }} - {% expression %}

Code

server.py

```
return render_template("index.html",
phrase = "hello", times = 5)
```

index.html

→  {{ prase }} 

→ {% for i in range (0, times): %}
{{ phrase }} 
{% endfor %}

→ {% if phrase == "hello" %}
 The prase says hello 
{% endif %}

```
boxes.foreach((box) => {
box.style.bgColor = color
```

## Another example:

```
@app.route('/play/<int:number>')
def play2(number):
  return render_template('play2.html',
    number = number)
```

number = number

- is the name variable you're passing to the template, and inside play2.html you can access it as number
- the python func parameter, coming from the URL

## NOTE

number = number

→ the first one is the name variable you're passing to the template, and inside play3.html you can access it as number

→ The second one is the python function parameter, coming from the url

## Another example that uses JS:

```
@app.route('/play/<int:number>/<color>')
def play3(number, color):
    return render_template("play3.html",
      number= number, color = color)
```

HTML

```
{% for i in range (number): %}
  <div class="box"></div>
{% endfor %}
```

```
<script>
  var boxes = - -
  var color = "{{ color }}"
```

# Static Files

## Static Content:

Refers to any content that can be served to the client without being modified, generated, or processed by the server.

---

## In Flask

→ Create static directory
→ it will serve css / js / images

---

## Organization:

→ hello.py
→ static
  → css
    → my-css.css
  → js
    → my-script.js
  → img
    → my-img.js
→ templates
  → index.html

---

## Code

→ <link rel="stylesheet" type="text/css"
href="{{ url_for('static', filename="mycss.css")}}"

# More Template Rendering:

## passing lists to Jinja:

hello_flask / hello.py

```
@app.route('/lists')
def render_lists():
    student_info = [
        {'name': "Michael", "age": 35},
        {'name': 'John', age: 30}
    ]

return render_template("list.html", random_numb
    = [3, 1, 5], students = student_info)
```

---

## HTML

```
{% for i in random_numbers %}
    <p> {{ number }} </p>
{% endfor %}

{% for students in students %}
    <p> {{ student['name'] }},
    {{ student['age'] }} </p>
{% endfor %}
```

# Post From Submission

How does the server take information from the user?

Using Forms

* Critical parts of the Form:

1. Action attribute:

This route will process the form

2. Method attributes:

POST, GET

3. Input Elements:

- gathering information about the user.
- each component should have a unique value for its name value.

4. A way to submit the form:

```
<input type='submit'>
<button> Submit </button>
```

NOT → `<input type="button">`

* What should happen when the form is submitted:

form_test/server.py

```
@app.route('/users', methods=['POST'])
def create_user():
    name_from_form = request.form['name']
    email_from_form = request.form['email']
    return render_template("show.html",
name_on_template = name_from_form,
name_on_template = email_fromfrom)
```

To the HTML

---

* Specifying allowed methods: methods=['POST']

- The default is Get
- It's a list and it can provide more than one value

---

* Accessing Data: request.form['name_of_input']

- The name of each html element has to be significant.

- On the server, we can access user data input through request.form.

returns a string.

---

* Display on HTML:

```
<p> Name: {{ name_on_template }} </p>
<p> Email: {{ email_on_template }} </p>
```

# Redirecting

Code

server.py:

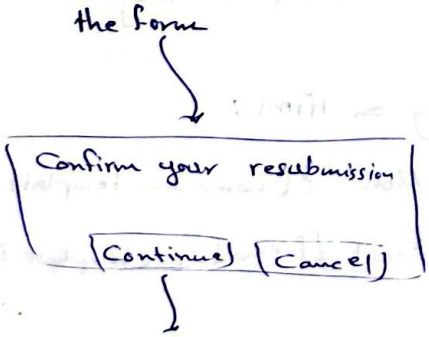
```
@app.route('/')
def index():
    return render_template("index.html")

@app.route("/users", methods = ['POST'])
def redirect():
    name_from_form = request.form['name']
    email_from_form = request.form['email']

    return render_template("show.html",
    name_on_template = name_from_form,
    email_on_template = email_from_form)
```

---

Rendering method: should always be used to prevent data from being processed more than once

You should notice that in the previous ex when you refresh the show.html page it will send a message indicating that the same form data is being resubmitted for processing

When you refresh the page after resbumitting the form




It will add the same user to database again

# Handling the rerendering:

```
def create_user('/user', methods = ['POST'])
    name = request.form['name']
    --
    return redirect("/show.html")

app.route("/show")
def show_user():
    return render_template("show.html")
```

# Hidden Inputs

- Hidden inputs are hidden from the user.
- Used to transfer information between differen pages.

- hidden input: is an ordinary element without visual representation in the rendered HTML.

- Syntax `<input type="hidden" name="action" value="register">`

---

Example

```
<form method="post" action="/process">
  <input type="hidden" name="which form"
      value="register">

<form method="post" action="/process">
  <input type="hidden" name="which-form"
      value="login">

if request.form['which-form'] == "register":
    //do register here
```

# Session

* HTTP request /response cycle is stateless ⟹ meaning each cycle is independent and doesn't know about other requests before or after it.

* What is a Session?

With session we can establish a relationship with the client by saving valuable data for future use and reading data from previous cycles.

⟹ Used for presistent data storage.

The session helps with:

It bridges the gap between a stateless protocol like HTTP, and the stateful data generated.

---

Explanation:

1. HTTP is stateless:

Normally, when you send a request like loading a page and the server responds, that conversation ends.

The server forgets about you after sending the response.

2. State = Remembering across requests:

memory that has: what's in your cart, or if you're logged in.

→ State: The general concept of remembering information across requests (It's the actual memory itself)

→ Session: A tool / technique to store and manage state in web apps

• [7] one method to store that info across multiple requests

In Short:

- The HTTP/request/response cycle is stateless, and each request/response cycle is independent and ignorant of all other requests.

- Presistent data storage such as session, allows for maintaining state across multiple request/response cycles.

- Session data can store valuable information such as user login status, identity and previous user interactions with the website.

- Presistent data storage, including sessions and databases, helps bridge the gap between stateless protocols like HTTP and stateful data generated through them.

- Cookies store session data, they're not that secure, so sensitive data cannot be saved in them.